



TEXAS A&M UNIVERSITY - SAN ANTONIO

CSCI 2436 Data Structures

Spring 2026

Chapter 6 Stack Implementations



Objectives

- Implement the stack by using either a **linked list** or an **array**
- Compare and contrast the various implementations and their performance



Interface for the Stack (from Ch5)

```
/** An interface for the ADT stack. */
```

```
public interface StackInterface<T>
```

```
{
```

```
    /** Adds a new entry to the top of this stack.
```

```
        @param newEntry An object to be added to the stack. */
```

```
    public void push(T newEntry);
```

```
    /** Removes and returns this stack's top entry.
```

```
        @return The object at the top of the stack.
```

```
        @throws EmptyStackException if the stack is empty before the  
operation. */
```

```
    public T pop();
```

```
    /** Retrieves this stack's top entry.
```

```
        @return The object at the top of the stack.
```

```
        @throws EmptyStackException if the stack is empty. */
```

```
    public T peek();
```

```
    /** Detects whether this stack is  
empty.
```

```
        @return True if the stack is empty.  
*/
```

```
    public boolean isEmpty();
```

```
    /** Removes all entries from this stack.  
*/
```

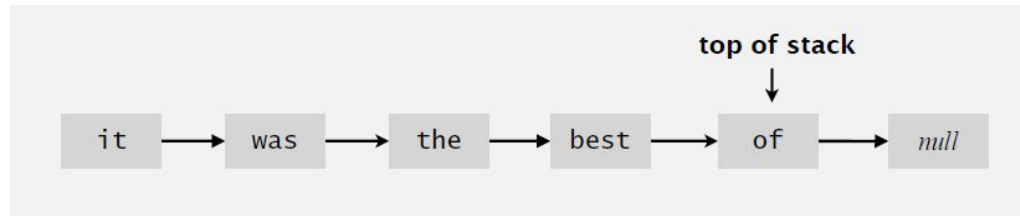
```
    public void clear();
```

```
} // end StackInterface
```

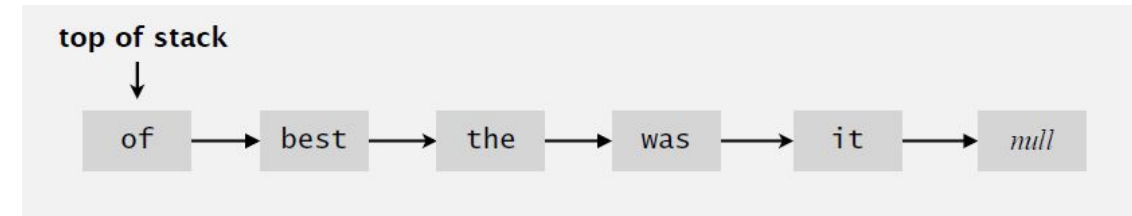


Linked-List Implementation

- Each operation involves **top of stack**
 - push
 - pop
 - peek
- Push "*it was the best of*" onto a stack, which word is at the top of stack? Where should it be stored?



A

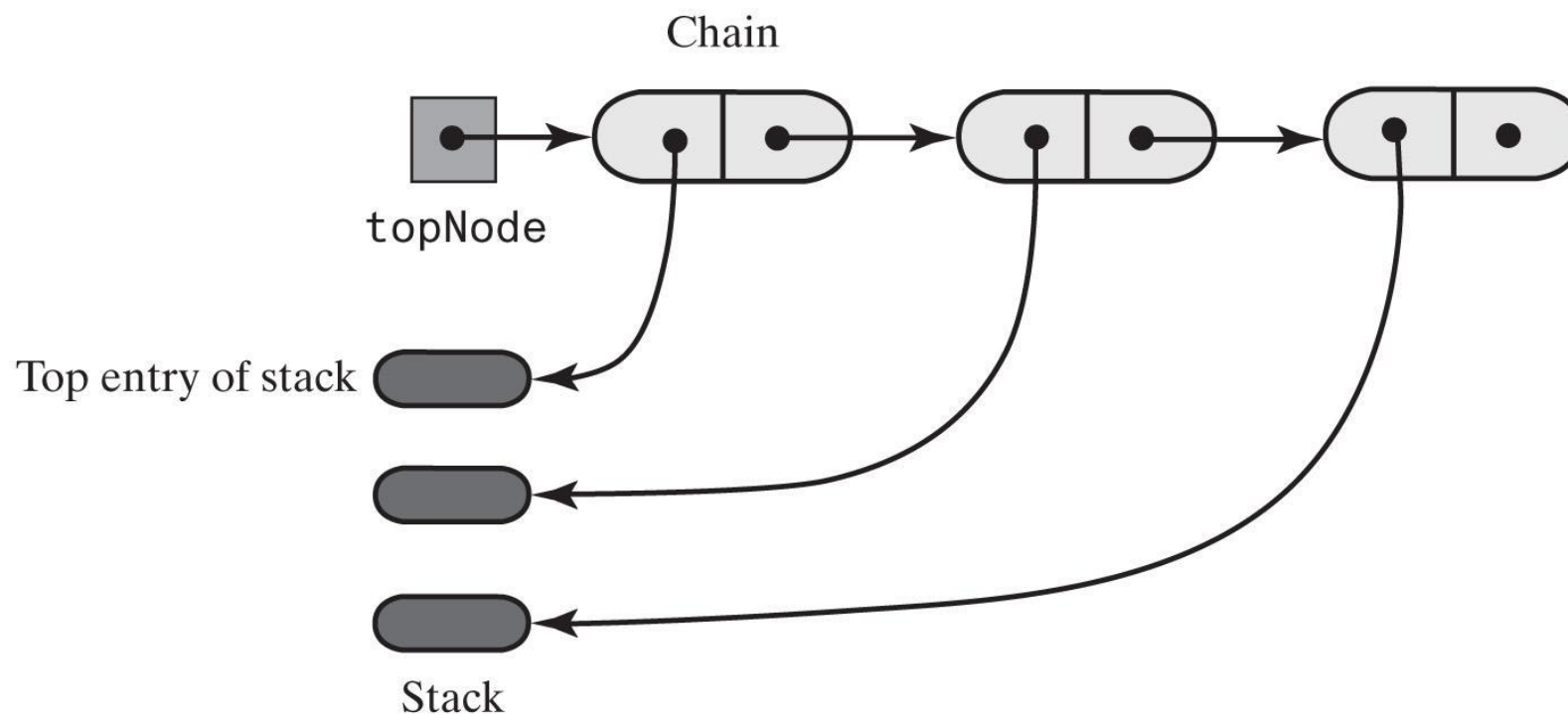


B



Linked-List Implementation

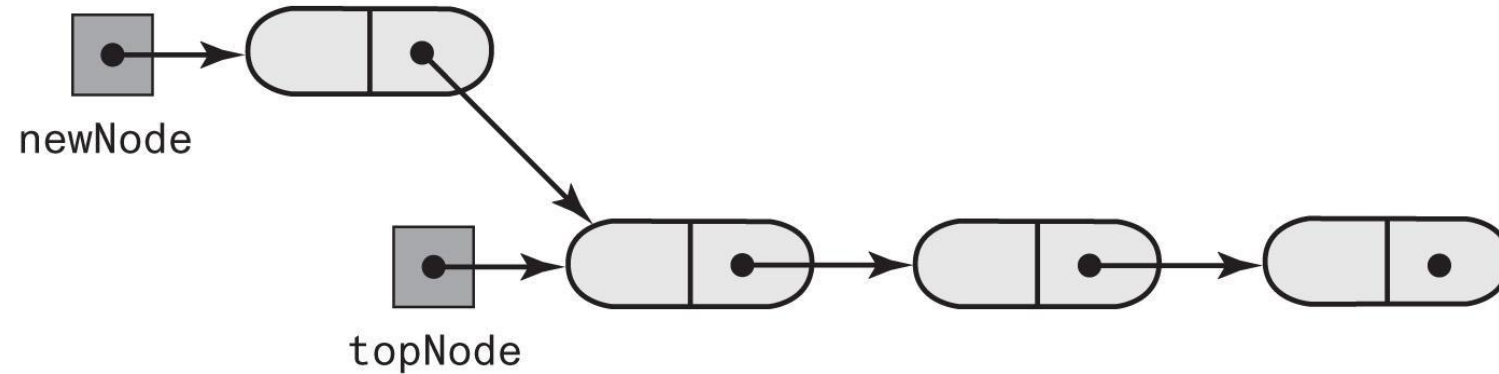
- A chain of linked nodes that implements a stack





Linked-List Implementation

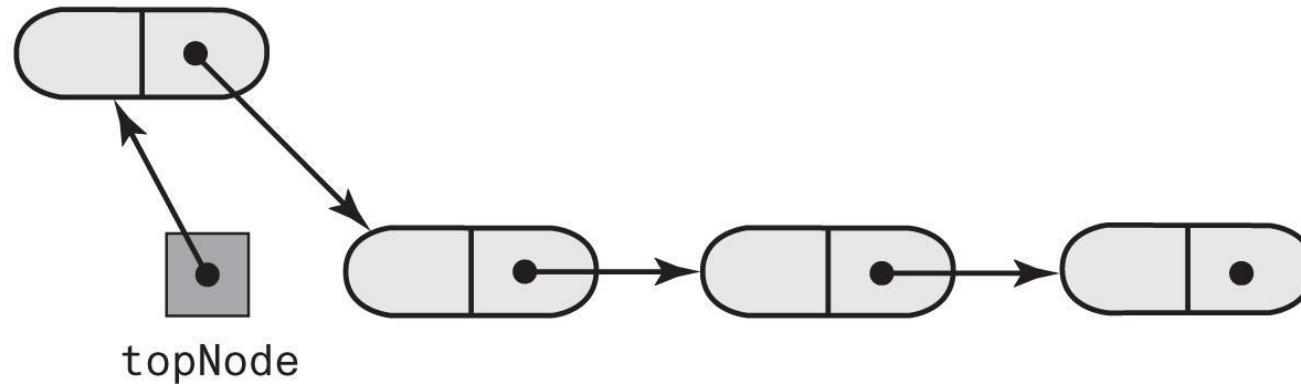
(a) A new node that references the node at the top of the stack



Adding a new node

© 2019 Pearson Education, Inc.

(b) The new node is now at the top of the stack

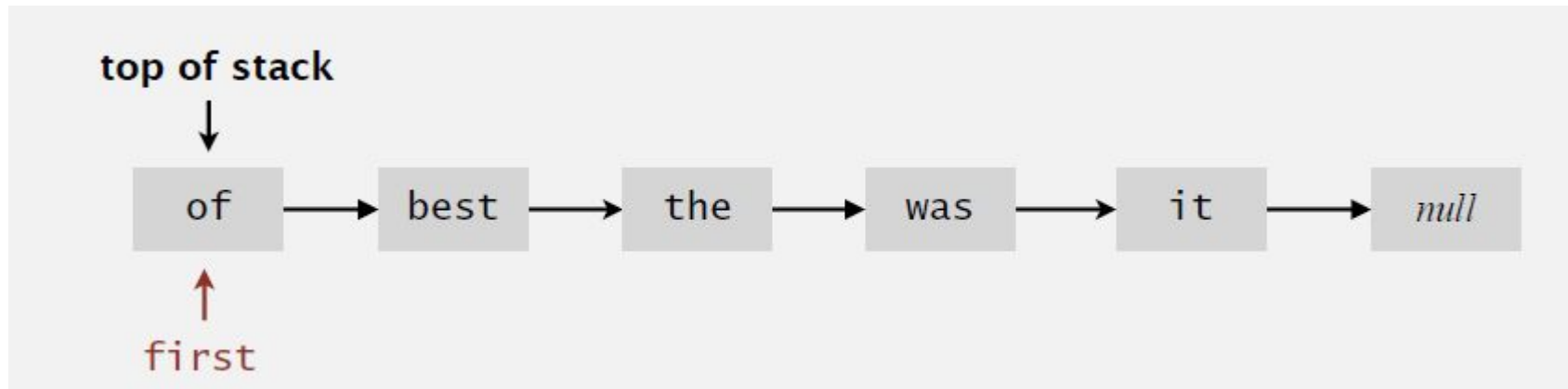


© 2019 Pearson Education, Inc.



Linked-List Implementation

- Maintain pointer **topNode (first)** to first node in a singly-linked list.
- Push new item before **topNode** .
- Pop item from **topNode** .





Linked-List Implementation

inner class

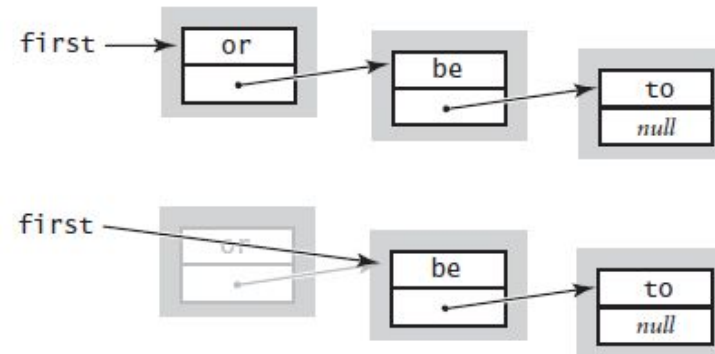
```
private class Node
{
    String item;
    Node next;
}
```

save item to return

```
String item = first.item;
```

delete first node

```
first = first.next;
```



return saved item

```
return item;
```




Linked Implementation of a Stack

- push

Step 1: Create a new node with **newEntry** and the link stored in **topNode**

Step 2: Save the link to the newNode to **topNode**

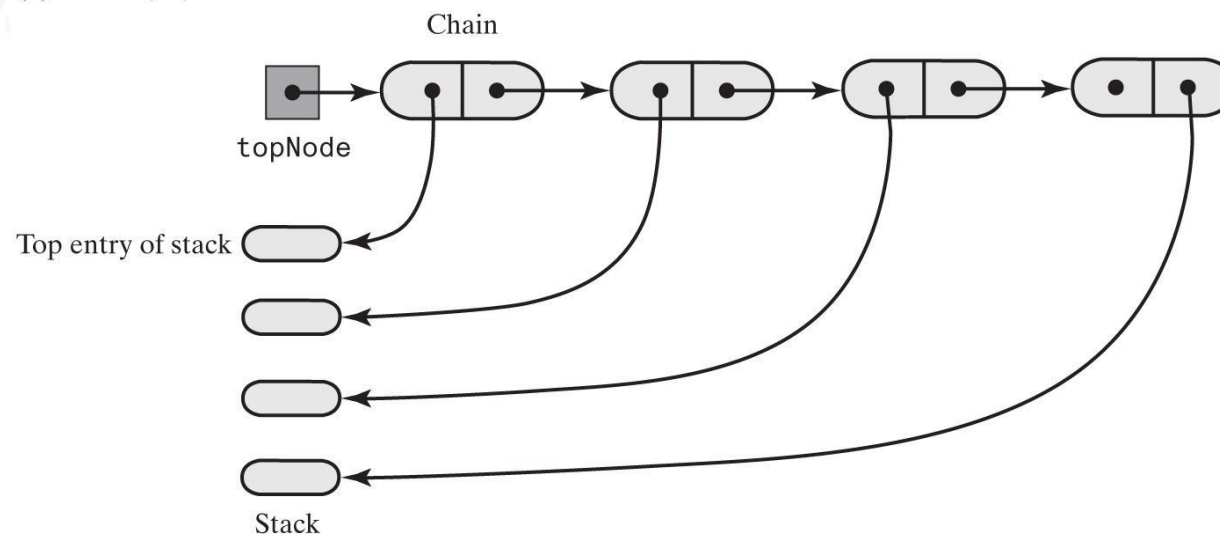
```
public void push(T newEntry)
{
    Step 1    Node newNode = new Node(newEntry, topNode);
    Step 2    topNode = newNode;
} // end push
```



Linked-List Implementation

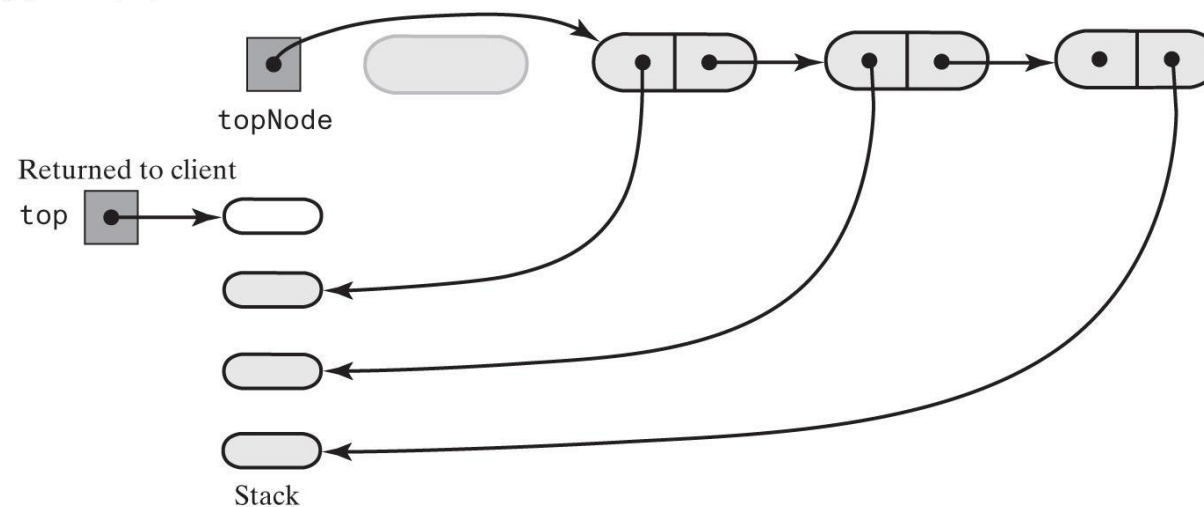
- The stack before and after pop deletes the first node in the chain

(a) Before pop



© 2019 Pearson Education, Inc.

(b) After pop



© 2019 Pearson Education, Inc.



Linked Implementation

```
/** A class of stacks whose entries are stored in a chain of nodes. */  
public final class LinkedStack<T> implements StackInterface<T>  
{  
    private Node topNode; // References the first node in the chain  
  
    public LinkedStack()  
    {  
        topNode = null;  
    } // end default constructor  
  
    // < Implementations of the stack operations go here. >  
    private class Node  
    {  
        private T data; // Entry in stack  
        private Node next; // Link to next node  
        // < Implementations of the node operations go here. >  
    } // end Node  
} // end LinkedStack
```

- An outline of a linked implementation of the ADT stack



Linked Implementation of a Stack

```
public T peek()
{
    if (isEmpty())
        throw new EmptyStackException();
    else
        return topNode.getData();
} // end peek
```

- peek and pop

```
public T pop()
{
    T top = peek(); // Might throw EmptyStackException

    // Assertion: topNode != null
    topNode = topNode.getNextNode();

    return top;
} // end pop
```



Linked Implementation of a Stack

- **isEmpty** and **clear**.

```
public boolean isEmpty()  
{  
    return topNode == null;  
} // end isEmpty
```

```
public void clear()  
{  
    topNode = null;  
} // end clear
```



Linked Implementation of a Stack

- Time Complexity Summary

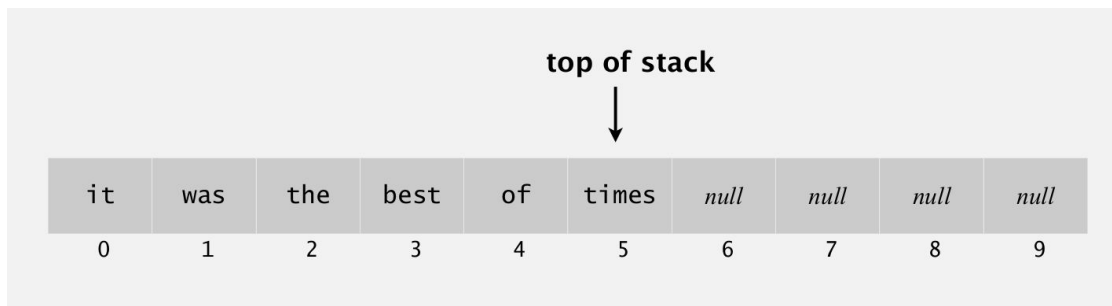
- push
- peek
- pop
- clear
- isEmpty

$O(1)$

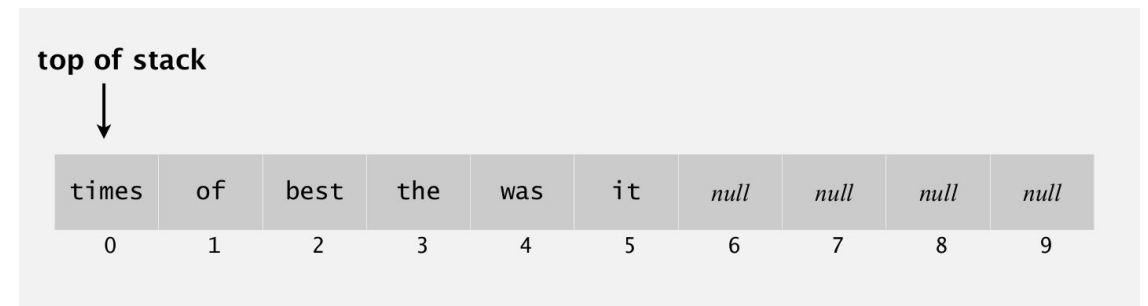


Array-Based Stack Implementation

- Each operation involves **top of stack**
 - **push**
 - **pop**
 - **peek**
- Push “*it was the best of*” onto a stack, which word is at the top of stack?
Where should it be stored?



A

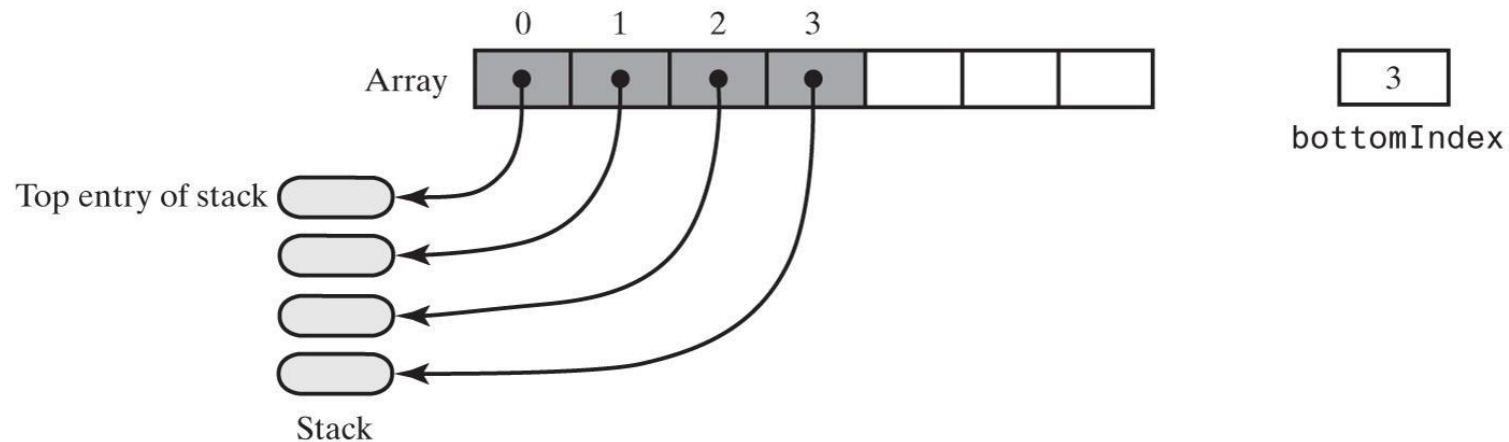


B



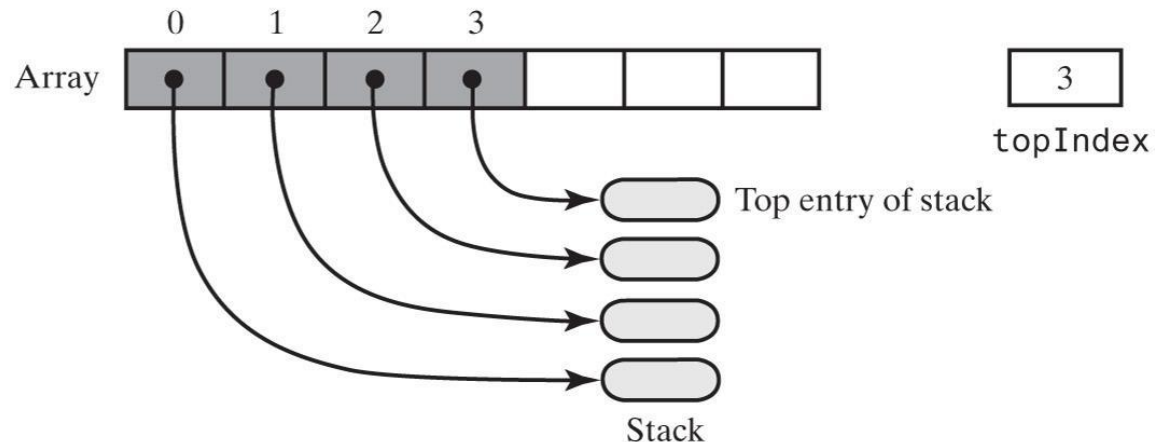
Array-Based Stack Implementation

(a) Inefficient: The array's first element references the stack's top entry



© 2019 Pearson Education, Inc.

(b) Efficient: The array's first element references the stack's bottom entry



© 2019 Pearson Education, Inc.

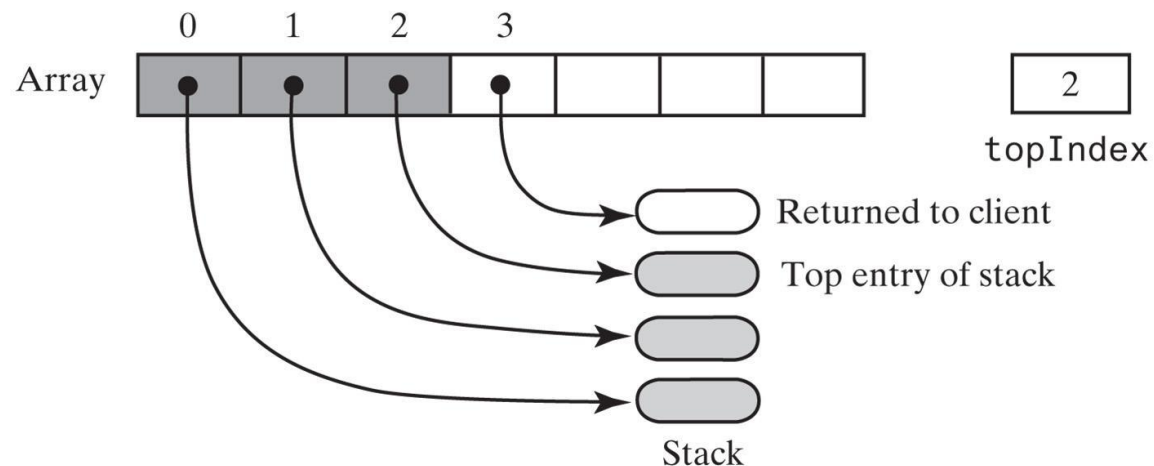
Keep **top of stack** at the **end of the array** – easiest to access



Array-Based Stack Implementation

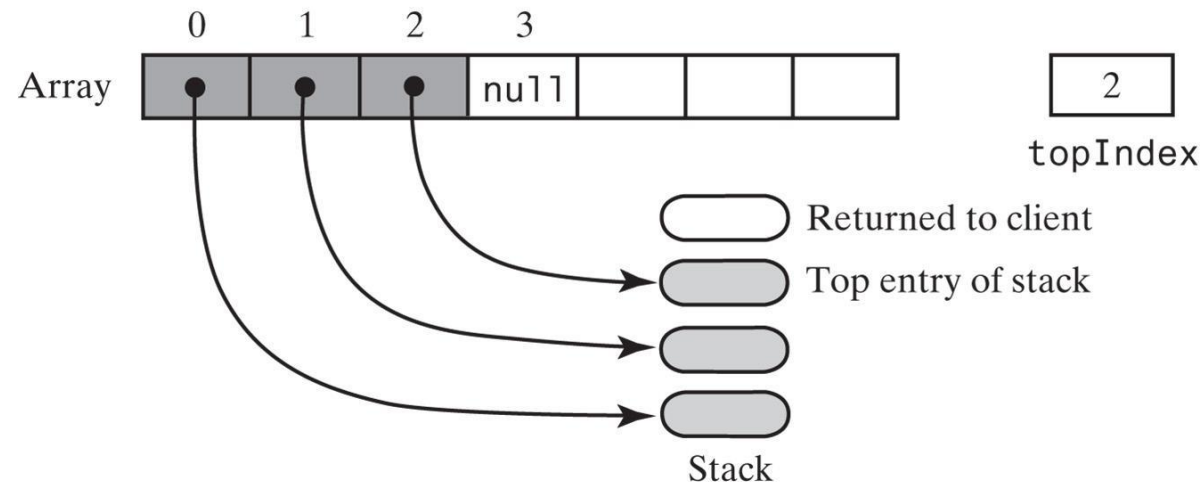
- An array-based stack after its top entry is removed in two different ways

(a) By decrementing topIndex



© 2019 Pearson Education, Inc.

(b) By setting `stack[topIndex]` to null and then decrementing `topIndex`



© 2019 Pearson Education, Inc.



Array-Based Stack Implementation

- An outline of an array-based implementation of the ADT stack

```
/** A class of stacks in an array. */
```

```
public final class ArrayStack<T> implements  
StackInterface<T>  
{  
    private T[] stack;    // Array of stack entries  
    private int topIndex; // Index of top entry  
    private static final int DEFAULT_CAPACITY = 50;  
    private static final int MAX_CAPACITY = 10000;
```

```
    public ArrayStack()  
    {  
        this(DEFAULT_CAPACITY);  
    } // end default constructor
```

```
public ArrayStack(int initialCapacity)
```

```
{  
    checkCapacity(initialCapacity);
```

```
    // safe cast because the new array contains null  
    @SuppressWarnings("unchecked")
```

```
    T[] tempStack = (T[])new Object[initialCapacity];  
    stack = tempStack;  
    topIndex = -1;
```

```
    } // end constructor
```

```
    // < Implementations of the stack operations go here.
```

```
>
```

```
    // ...
```

```
} // end ArrayStack
```



Array-Based Stack Implementation

```
public void push(T newEntry)
{
    ensureCapacity();
    stack[topIndex + 1] = newEntry;
    topIndex++;
} // end push

private void ensureCapacity()
{
    if (topIndex >= stack.length - 1) // If array is full, double its size
    {
        int newLength = 2 * stack.length;
        checkCapacity(newLength);
        stack = Arrays.copyOf(stack, newLength);
    } // end if
} // end ensureCapacity
```



Array-Based Stack Implementation

- Retrieving the top, operation is $O(1)$

```
public T peek()
{
    if (isEmpty())
        throw new EmptyStackException();
    else
        return stack[topIndex];
} // end peek
```

```
public T pop()
{
    if (isEmpty())
        throw new EmptyStackException();
    else
    {
        T top = stack[topIndex];
        stack[topIndex] = null;
        topIndex--;
        return top;
    } // end if
} // end pop
```



Linked vs Array-Based Stack Implementation

- Linked:
 - Each stack operation costs $O(1)$
 - Allocate node as needed, none is wasted.
 - Array-Based:
 - Each stack operation costs $O(1)$
 - Except when the array must be resized: the push operation causing resizing will cost $O(n)$. Average cost of push?
 - Possible wasted space.
-

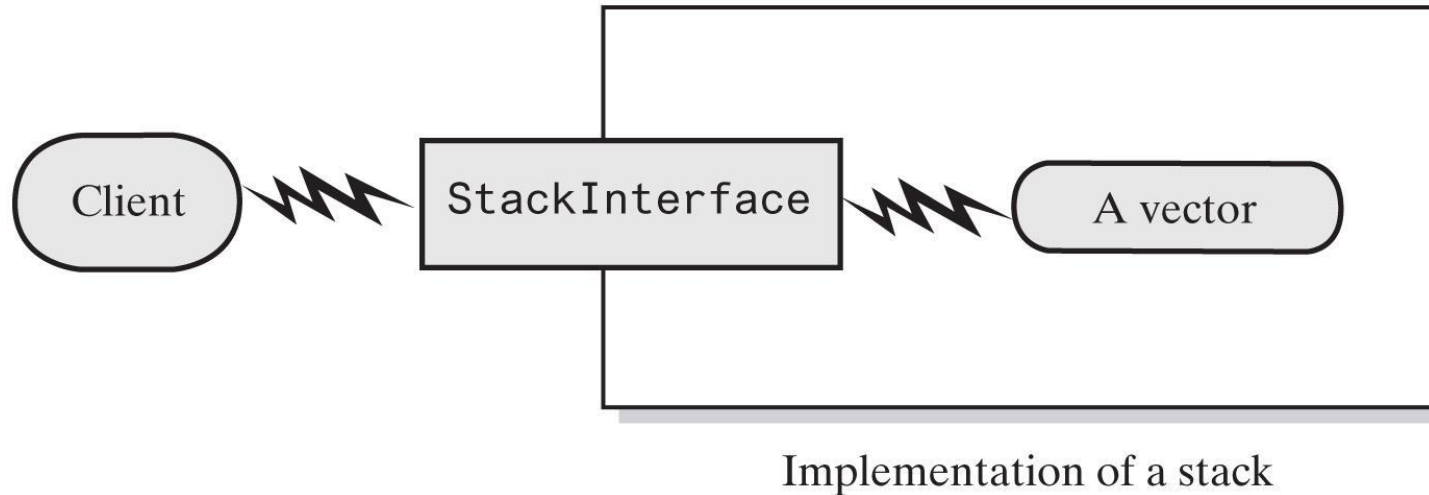


Vector-Based Stack Implementation

- The class **Vector**
 - An object that behaves like a high-level array
 - ✓ Index begins with 0
 - ✓ Methods to access or set entries
 - ✓ Size will grow as needed
 - Has methods to add, remove, clear
 - Also methods to determine
 - ✓ Last element
 - ✓ Is the vector empty
 - ✓ Number of entries
 - Use vector's methods to manipulate stack
-



Vector-Based Stack Implementation



© 2019 Pearson Education, Inc.

- A client using the methods given in **StackInterface**; these methods interact with a **vector's methods** to perform **stack operations**
- Java's class **Vector** uses an **array** in its implementation. It uses a **resizable** array to contain the stack's entries, and so the stack can **grow in size** as needed.



Vector-Based Stack Implementation

```
import java.util.Vector;
/** A class of stacks whose entries are stored in a vector. */
public final class VectorStack<T> implements StackInterface<T>
{
    private Vector<T> stack; // Last element is the top entry in stack
    private boolean integrityOK;
        private static final int DEFAULT_CAPACITY = 50;
        private static final int MAX_CAPACITY = 10000;

    public VectorStack()
    {
        this(DEFAULT_CAPACITY);
    } // end default constructor

    public VectorStack(int initialCapacity)
    {
        integrityOK = false;
        checkCapacity(initialCapacity);
        stack = new Vector<>(initialCapacity); // Size doubles as needed
        integrityOK = true;
    } // end constructor
```



Vector-Based Stack Implementation

```
public void push(T newEntry)
{
    checkIntegrity();
    stack.add(newEntry);
} // end push

public T peek()
{
    checkIntegrity();
    if (isEmpty())
        throw new EmptyStackException();
    else
        return stack.lastElement();
} // end peek
```



Vector-Based Stack Implementation

```
public T pop()
{
    checkIntegrity();
    if (isEmpty())
        throw new EmptyStackException();
    else
        return stack.remove(stack.size() - 1);
} // end pop

public boolean isEmpty()
{
    checkIntegrity();
    return stack.isEmpty();
} // end isEmpty

public void clear()
{
    checkIntegrity();
    stack.clear();
} // end clear
```
